

HTTP Security Headers

10

HEADERS

Every pentest report checks these.
Most sites are missing half of them.

01 HSTS

02 X-Content-Type

03 X-Frame-Options

04 Referrer-Policy

05 Permissions-Policy

06 CSP

07 Cross-Origin-*

08 Set-Cookie

09 Clear-Site-Data

10 Remove headers

swipe →



@kerem.tech



@keraattin



HSTS

01 / 10

Strict-Transport-Security

The screenshot shows the Chrome DevTools Network tab with the 'Network' panel selected. The search bar contains 'strict-transport'. The filter is set to 'Doc'. The selected resource is 'github.com' with a status of 200 OK. The 'Headers' panel is expanded, showing the 'Response Headers' section. The 'strict-transport-security' header is highlighted, showing the following values: 'max-age=31536000;', 'includeSubDomains;', and 'preload'. Other visible headers include 'content-type: text/html; charset=utf-8', 'server: GitHub.com', and 'cache-control: no-cache'.

```
strict-transport-security:
  max-age=31536000;
  includeSubDomains;
  preload

content-type: text/html; charset=utf-8
server: GitHub.com
cache-control: no-cache
```

Directives

- **max-age=31536000** · cache duration in seconds · 1 year
- **includeSubDomains** · applies to all subdomains
- **preload** · eligible for browser preload list

What it prevents

Stops MITM attacks via SSL stripping. Once seen, the browser auto-upgrades every HTTP request to HTTPS for the configured duration. Without it, attackers on public WiFi can intercept traffic before the redirect fires · add preload for the first visit.



@kerem.tech



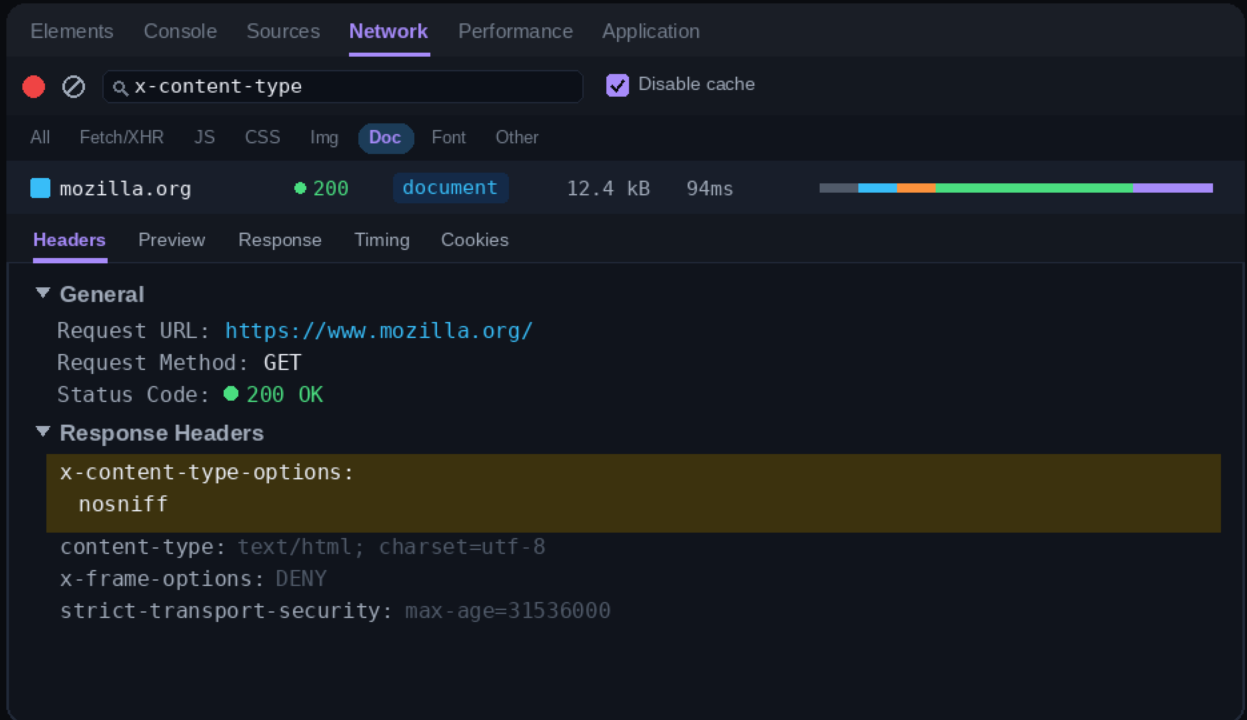
@keraattin



X-Content-Type

02 / 10

X-Content-Type-Options



Directives

→ **nosniff** · the only valid value · disables MIME-type sniffing

What it prevents

Stops MIME-type sniffing attacks. Some browsers historically guessed content types, ignoring the declared Content-Type. Attackers exploited this by uploading files with safe extensions that contained executable payloads (a .jpg that was actually JS).



@kerem.tech



@keraattin



X-Frame-Options

03 / 10

X-Frame-Options

The screenshot shows the Network tab of a web browser. The search bar contains 'x-frame'. The selected resource is 'accounts.google.com' with a status of 200 OK. The 'Doc' filter is active. The 'Headers' tab is selected, showing the 'Response Headers' section. The 'x-frame-options' header is highlighted with a yellow background and shows the value 'DENY'. Other headers visible include 'content-type: text/html; charset=utf-8', 'strict-transport-security: max-age=31536000', and 'x-content-type-options: nosniff'.

```
Request URL: https://accounts.google.com/
Request Method: GET
Status Code: 200 OK

Response Headers:
x-frame-options: DENY
content-type: text/html; charset=utf-8
strict-transport-security: max-age=31536000
x-content-type-options: nosniff
```

Directives

- **DENY** · no site can frame this page · safest default
- **SAMEORIGIN** · only same origin can frame · use if you self-frame
- **ALLOW-FROM** · deprecated · migrate to CSP frame-ancestors

What it prevents

Defends against clickjacking. Without this, attackers can embed your page in an invisible iframe overlaid on fake UI, tricking users into clicking what looks legitimate but performs actions on your authenticated site · CSP frame-ancestors supersedes it.



@kerem.tech



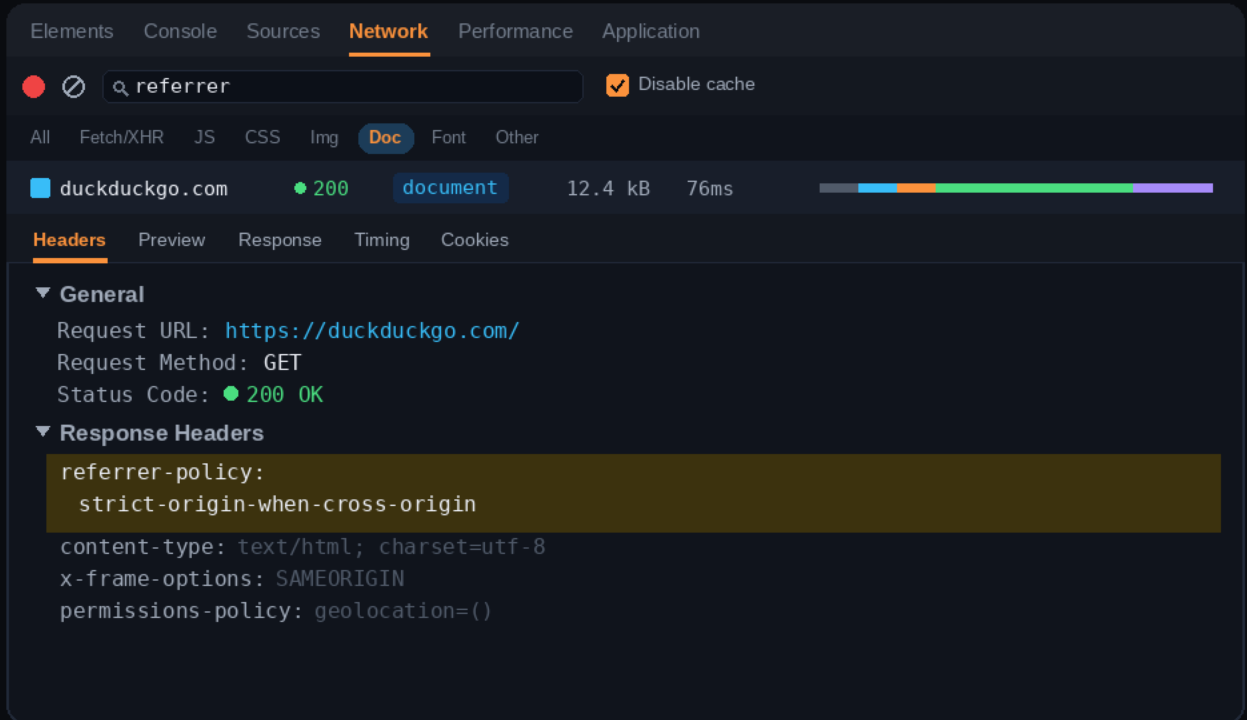
@keraattin



Referrer-Policy

04 / 10

Referrer-Policy



Directives

- **no-referrer** · never send the Referer header
- **strict-origin** · send origin over HTTPS only · no path
- **strict-origin-when-cross-origin** · modern default · privacy + UX balance

What it prevents

Stops information leakage via the Referer header. Without this, your URLs (which may carry session tokens, user IDs, search queries, or internal paths) are sent to every external site you link to or load resources from · a privacy and security leak.



@kerem.tech



@keraattin



Permissions

05 / 10

Permissions-Policy

Elements Console Sources **Network** Performance Application

permissions Disable cache

All Fetch/XHR JS CSS Img **Doc** Font Other

github.com 200 document 12.4 kB 118ms

Headers Preview Response Timing Cookies

General

Request URL: <https://github.com/>
Request Method: GET
Status Code: 200 OK

Response Headers

permissions-policy:
camera=(), microphone=(),
geolocation=(), payment=()

content-type: text/html; charset=utf-8
x-frame-options: DENY
strict-transport-security: max-age=31536000

Directives

- **camera=()** · disables camera access entirely
- **microphone=()** · disables microphone access
- **geolocation=()** · disables geolocation API
- **payment=()** · disables the Payment Request API

What it prevents

Locks down powerful browser APIs your site doesn't need · camera, microphone, geolocation, payment, USB, sensors. Without this, a compromised script or third-party iframe can prompt the user for these capabilities · even a denied prompt is a phishing surface.



@kerem.tech



@keraattin



CSP

06 / 10

Content-Security-Policy

The screenshot shows a browser's developer tools with the Network tab selected. A search filter 'content-security' is applied. The selected resource is a document from github.com with a status of 200 OK. The 'Headers' sub-tab is active, showing the 'Response Headers'. The 'content-security-policy' header is highlighted, displaying the following directives: default-src 'self'; script-src 'self' cdn.example.com; frame-ancestors 'none'; object-src 'none'. Other visible headers include content-type: text/html; charset=utf-8, x-frame-options: DENY, and strict-transport-security: max-age=31536000.

```
content-security-policy:
  default-src 'self';
  script-src 'self' cdn.example.com;
  frame-ancestors 'none';
  object-src 'none'

content-type: text/html; charset=utf-8
x-frame-options: DENY
strict-transport-security: max-age=31536000
```

Directives

- **default-src 'self'** · fallback for all resource types
- **script-src** · whitelist script sources · defeats XSS
- **frame-ancestors 'none'** · modern clickjacking defense
- **object-src 'none'** · block Flash, plugins, embeds

What it prevents

The most powerful XSS defense. By whitelisting which sources can load scripts, styles, and frames, CSP defeats most injection payloads · even if an attacker injects `<script>`, it won't run. Start with Content-Security-Policy-Report-Only to find violations.



@kerem.tech



@keraattin



Cross-Origin-*

07 / 10

COOP · COEP · CORP

The screenshot shows the Chrome DevTools Network tab. The search bar contains 'cross-origin'. The selected resource is a document from figma.com with a status of 200 OK. The 'Headers' tab is active, showing the following response headers:

```
cross-origin-opener-policy: same-origin
cross-origin-embedder-policy: require-corp
cross-origin-resource-policy: same-site
content-type: text/html; charset=utf-8
strict-transport-security: max-age=31536000
x-frame-options: SAMEORIGIN
```

Directives

- **COOP: same-origin** · isolates browsing context from other origins
- **COEP: require-corp** · embedded resources must explicitly opt in
- **CORP: same-site** · controls who can load your resources

What it prevents

Defends against Spectre-class side-channel attacks and cross-origin data leaks. Without these, malicious origins use timing tricks and shared browser memory to read data from your origin. All three together enable cross-origin isolation · required for SharedArrayBuffer.



@kerem.tech



@keraattin



Set-Cookie

08 / 10

Cookie Security Flags

The screenshot shows the Network tab of a web browser. The search bar contains 'set-cookie'. The selected resource is 'stripe.com' with a status of 200 OK. The 'Headers' tab is active, showing the 'Response Headers' section. The 'set-cookie' header is highlighted, showing the value 'session=abc123def456; HttpOnly; Secure; SameSite=Strict; Path=/'. Other headers visible are 'content-type: application/json', 'strict-transport-security: max-age=63072000', and 'x-frame-options: DENY'.

Directives

- **HttpOnly** · JavaScript can't read this cookie · blocks XSS theft
- **Secure** · only sent over HTTPS · no plaintext leaks
- **SameSite=Strict** · never sent on cross-site requests · CSRF defense

What it prevents

Hardens cookies against three attacks. Without HttpOnly, an XSS payload reads `document.cookie` and exfiltrates the session. Without Secure, a single HTTP request leaks the cookie on the network. Without SameSite, malicious sites can trigger authenticated actions.



@kerem.tech



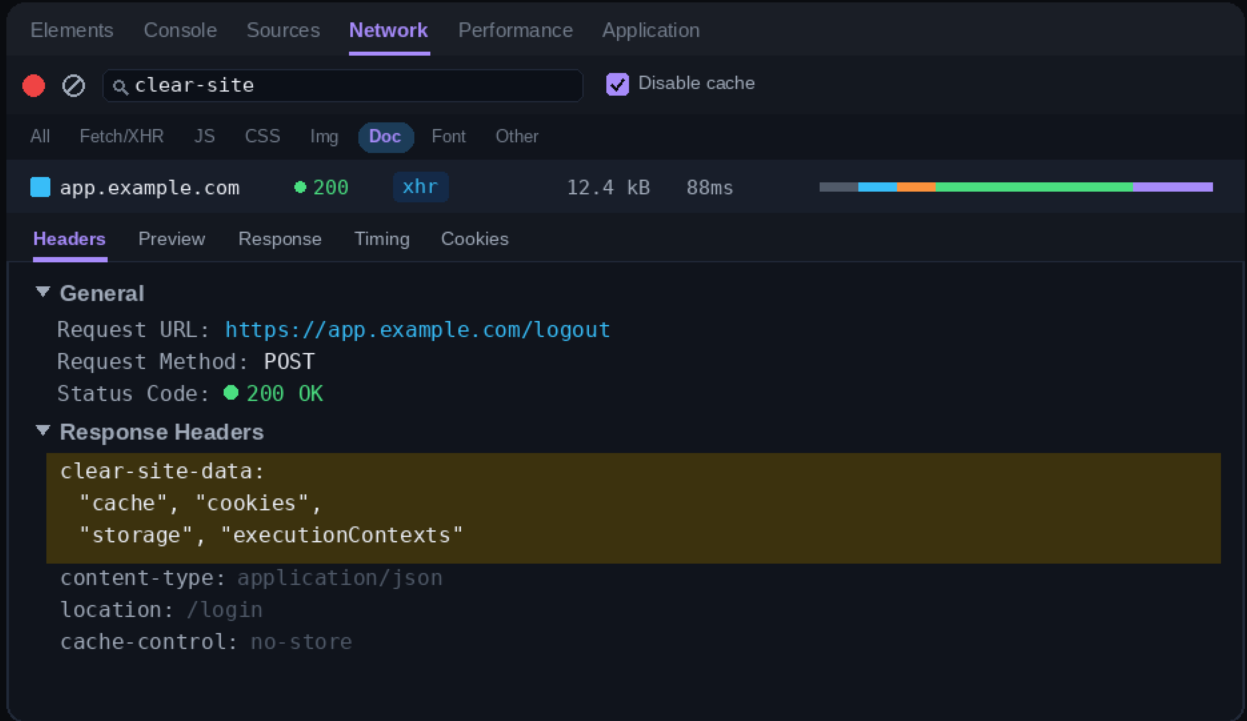
@keraattin



Clear-Site-Data

09 / 10

Clear-Site-Data



Directives

- **"cache"** · wipes the browser cache for this origin
- **"cookies"** · removes all cookies for the origin
- **"storage"** · clears localStorage, sessionStorage, IndexedDB
- **"executionContexts"** · reloads pages so cleared state takes effect

What it prevents

Ensures clean logout. Without this, signing out doesn't actually clear sensitive data · cookies, localStorage tokens, IndexedDB caches, cached pages with PII can persist after the session ends. Set this header on the logout response to wipe everything at once.



@kerem.tech



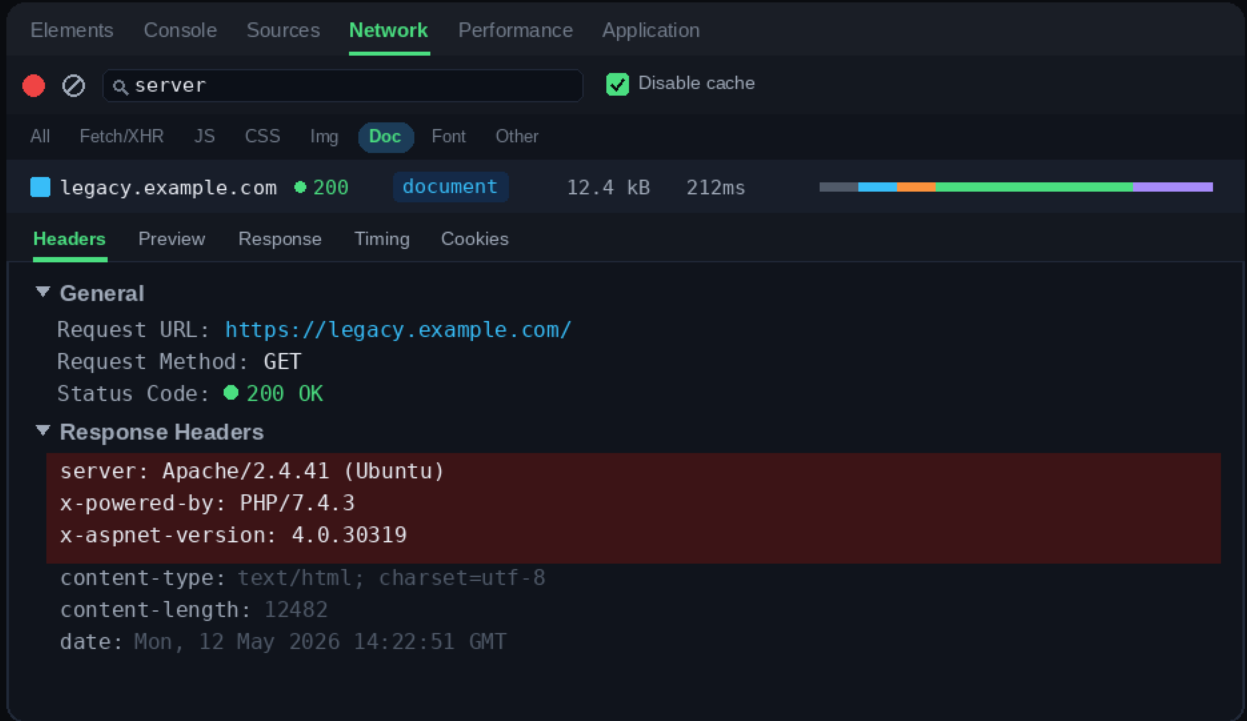
@keraattin



Remove These

10 / 10

Information Disclosure Headers



Directives

- **Server** · leaks web-server name and exact version
- **X-Powered-By** · leaks language and framework version
- **X-AspNet-Version** · leaks .NET runtime version
- **X-AspNetMvc-Version** · leaks MVC framework version

What it prevents

Server, X-Powered-By, and friends leak your tech stack version to anyone who sends a request. Attackers use this to look up known CVEs for your exact version and skip reconnaissance entirely · even if patched today, tomorrow's CVE has your version on every response.



@kerem.tech



@keraattin



That's all 10 headers.



Audit your site today.
Most are 5 minutes to add.

→ **SAVE**
for your next security audit

→ **SHARE**
send to your dev / sec team



@kerem.tech



@keraattin



@keraattin



@keraattin
Kerem Tech



@kerem.tech



@keraattin

